In [2]:
```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_4b_bottlenecked"
model = VGG_quant(vgg_name = 'VGG16_4b_bottlenecked', act_bit=4)

# print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.2


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_si


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))
```

```
testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch inc
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()
```

```
    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()
```

```
    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 12
    adjust_list = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 1
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.9

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
=> Building model...
```

In [ ]:

```
# This cell won't be given, but students will complete the training

lr = 0.01

weight_decay = 5e-4
momentum = 0.9
epochs = 200
best_prec = 0

#model = nn.DataParallel(model).cuda()
model.cuda()
criterion = nn.CrossEntropyLoss().cuda()
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_
#cudnn.benchmark = True

if not os.path.exists('result'):
    os.makedirs('result')
fdir = 'result/'+str(model_name)
if not os.path.exists(fdir):
    os.makedirs(fdir)
```

```
for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec,best_prec)
    print('best acc: {:1f}'.format(best_prec))
    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)
```

In [3]:

```
PATH = "result/VGG16_quant/4b_best_pth.tar"
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 9180/10000 (92%)
```

In [4]:

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []
```

```
save_output = SaveOutput()
```

In [5]:
```
#send an input and grap the value by using prehook like HW3
for idx, layer in enumerate(model.modules()):
    if isinstance(layer, QuantConv2d):
        # print("prehooked:", idx)
        # print(layer)
        layer.register_forward_pre_hook(save_output)

# we care about -5
```

In [6]:
```
dataiter = iter(testloader)
images, labels = next(dataiter)
images = images[0].unsqueeze(0).cuda()
output = model(images)
```

In [7]:
```
print(save_output.outputs[-5][0].shape)
print(save_output.outputs[-4][0].shape)
# print(save_output.outputs[-5][0])
# print(save_output.outputs[-4][0])
```

```
torch.Size([1, 8, 4, 4])
torch.Size([1, 8, 4, 4])
```

In [8]:
```
w_bit = 4
weight_q = model.features[27].weight_q # quantized value is stored during th
w_alpha = model.features[27].weight_quant.wgt_alpha  # alpha is defined in y
w_delta = (w_alpha / (2**(w_bit-1)-1))    # delta can be calculated by using
weight_int = weight_q / w_delta # w_int can be calculated by weight_q and w_
# print(weight_int) # you should see clean integer numbers
print(weight_int.shape)
```

```
torch.Size([8, 8, 3, 3])
```

In [9]:
```
act = save_output.outputs[-5][0]
act_alpha  = model.features[27].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
# print(act_int)
```

In [10]:
```
conv_int = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3,
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
conv_int.bias = model.features[27].bias
output_int = conv_int(act_int)
# print(output_int)
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2
# print(output_recovered)
```

In [11]:
```
## This cell is provided
```

```
conv_ref = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3,
conv_ref.weight = model.features[27].weight_q
conv_ref.bias = model.features[27].bias
output_ref = conv_ref(act)
#print(output_ref)

print(abs((output_ref - output_recovered)).mean())
```

```
tensor(0.0845, device='cuda:0', grad_fn=<MeanBackward0>)
```

In [12]:

```
act_int_single = act_int[0, ...] # take only first of the batch
C, H, W = act_int_single.shape
p = 1

a_pad = torch.zeros((C, H + 2 * p, W + 2 * p))
a_pad[:, p:p+H, p:p+W] = act_int_single

a_pad_flat = torch.reshape(a_pad, (a_pad.size(0), -1))
a_pad_nij = torch.permute(a_pad_flat, (1,0))

# print(act_int_single)
# print(a_pad)

print(a_pad.shape)

# print(a_pad_nij)
print(a_pad_nij.shape)

w_int_kij = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1
# print(w_int_kij)
print(w_int_kij.shape)

output_int_single = output_int[0, ...]
out_int_flat = torch.reshape(output_int_single, (output_int_single.size(0),
out_int_nij = torch.permute(out_int_flat, (1, 0))
# apply reLU
out_int_nij.clamp_(min=0)
```

```
torch.Size([8, 6, 6])
torch.Size([36, 8])
torch.Size([8, 8, 9])
```

Out[12]:
```
tensor([[ 58.,  74.,   6.,   0.,   0.,  22.,   5.,   0.],
        [ 22., 172., 155.,   0.,   0., 111.,   0.,   0.],
        [139., 193., 105.,   0.,   0., 236.,   0.,   0.],
        [  0., 141.,  78.,   0.,   0.,  70.,   0.,  17.],
        [178., 104.,  71.,   0.,   0.,  34., 123.,   0.],
        [177., 177., 178.,   0.,   0., 121., 192.,   0.],
        [234., 149.,   2.,   0.,   0., 149., 143.,   0.],
        [ 65.,  50.,   0.,   0.,  41.,  75.,  95.,   0.],
        [204.,   0.,  98.,   0.,   0.,   0., 101.,  89.],
        [280.,  35., 202.,   0.,   0.,   0., 125., 106.],
        [352.,  22.,  42.,   0.,   0.,   0.,   0., 143.],
        [160.,  10.,  21.,  13.,   0.,   0.,  26., 103.],
        [ 91.,   0.,  44.,   7.,   0.,   0.,  20.,  87.],
        [179.,   0., 115.,   0.,   5.,   0.,  18., 100.],
        [159.,   0.,  61.,   0.,   0.,   0.,   0.,  88.],
        [ 29.,   0.,  48.,   0.,   0.,   0.,   0.,  40.]], device='cuda:0',
       grad_fn=<AsStridedBackward0>)
```

In [13]:
```
num_outputs = 8
bit_precision = 4

C, H_pad, W_pad = a_pad.shape

H_out = H
W_out = W

activations_os = []

for out_idx in range(num_outputs):
    row_out = out_idx // W_out
    col_out = out_idx % W_out

    output_vector = []

    for channel in range(C):
        for dr in range(3):
            for dc in range(3):
                r = row_out + dr
                c = col_out + dc

                act_val = a_pad[channel, r, c]
                output_vector.append(act_val)

    output_tensor = torch.tensor(output_vector)
    activations_os.append(output_tensor)

activations_os_tensor = torch.stack(activations_os)

activations_os_transposed = activations_os_tensor.t()

file = open('activation_os.txt', 'w')
file.write('#time0:out7_ch0_pos0,out6_ch0_pos0,...,out0_ch0_pos0#\n')
file.write('#time1:out7_ch0_pos1,out6_ch0_pos1,...,out0_ch0_pos1#\n')
file.write('#...72Rowstotal(8Channels×9KernelPositions)#\n')

for time_step in range(72):
```

```
    for out_idx in range(num_outputs - 1, -1, -1):
        act_val = activations_os_transposed[time_step, out_idx]
        X_bin = '{0:04b}'.format(round(act_val.item()))
        for k in range(bit_precision):
            file.write(X_bin[k])
    file.write("\n")

file.close()
```

```
/tmp/ipykernel_1876351/1712133493.py:26: UserWarning: Converting a tensor wi
th requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at /pytorch/torc
h/csrc/autograd/generated/python_variable_methods.cpp:836.)
  output_tensor = torch.tensor(output_vector)
```

In [14]:

```
weights_os = []

for input_channel in range(8):
    for kernel_pos in range(9):

        time_step_weights = []
        for output_channel in range(8):
            weight_val = w_int_kij[output_channel, input_channel, kernel_pos
            time_step_weights.append(weight_val)

        weights_os.append(torch.tensor(time_step_weights))

weights_os_tensor = torch.stack(weights_os)

file = open('weights_os.txt', 'w')
file.write('#time0:out7_ch0_pos0,out6_ch0_pos0,...,out0_ch0_pos0#\n')
file.write('#time1:out7_ch0_pos1,out6_ch0_pos1,...,out0_ch0_pos1#\n')
file.write('#72RowsTotal(8InputChannels×9KernelPositions)#\n')

bit_precision=4

for time_step in range(72):
    for out_channel in range(7, -1, -1):
        weight_val = weights_os_tensor[time_step, out_channel]
        val = int(round(weight_val.item()))

        if (val < 0):
            val = val + (2 ** bit_precision)

        W_bin = f'{val:0{bit_precision}b}'

        for k in range(bit_precision):
            file.write(W_bin[k])

    file.write("\n")

file.close()
```

In [16]:

```
output_os_computed = activations_os_transposed.T @ weights_os_tensor
output_os_computed = output_os_computed.cuda()
```

```
output_os_computed = output_os_computed.clamp(min=0)

output_reference = []
for out_idx in range(8):
    row_out = out_idx // W_out
    col_out = out_idx % W_out

    ref_val = output_int_single[:, row_out, col_out]
    output_reference.append(ref_val)

output_reference_tensor = torch.stack(output_reference)

difference = torch.abs(output_os_computed - output_reference_tensor)
print(f"\nAbsolute difference:\n{difference}")
print(f"Max difference: {difference.max().item()}")
print(f"Mean difference: {difference.mean().item()}")

if torch.allclose(output_os_computed, output_reference_tensor, atol=1e-5):
    print("\n✓ Output Stationary computation MATCHES reference output!")
else:
    print("\n✗ Output Stationary computation DOES NOT match reference output
```

```
Absolute difference:
tensor([[0.0000e+00, 0.0000e+00, 3.8147e-06, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         1.9073e-06, 0.0000e+00],
        [1.1444e-05, 1.5259e-05, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         0.0000e+00, 0.0000e+00],
        [1.5259e-05, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 3.8147e
-06,
         0.0000e+00, 0.0000e+00],
        [0.0000e+00, 1.5259e-05, 1.5259e-05, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 3.8147e-06, 0.0000e+00, 0.0000e+00, 0.0000e
+00,
         1.5259e-05, 0.0000e+00],
        [0.0000e+00, 7.6294e-06, 0.0000e+00, 0.0000e+00, 7.6294e-06, 0.0000e
+00,
         7.6294e-06, 0.0000e+00]], device='cuda:0', grad_fn=<AbsBackward0>)
Max difference: 1.52587890625e-05
Mean difference: 1.9371509552001953e-06

✓ Output Stationary computation MATCHES reference output!
```

In [17]:

```
file = open('output_os.txt', 'w')
file.write('#Row0:out7_element0,out6_element0,...,out0_element0#\n')
file.write('#Row1:out7_element1,out6_element1,...,out0_element1#\n')
file.write('#............#\n')
```

```
file.write('#Row1:out7_element7,out6_element7,...,out0_element7#\n')
file.write('#8RowsTotal(8OutputElements×8OutputChannels)#\n')

bit_precision = 16

for out_elem in range(8):
    for out_chan in range(7, -1, -1):
        val = int(output_reference_tensor[out_elem, out_chan].item())

        if (val < 0):
            val = val + (2 ** bit_precision)

        W_bin = f'{val:0{bit_precision}b}'

        for k in range(bit_precision):
            file.write(W_bin[k])

    file.write("\n")

file.close()
```